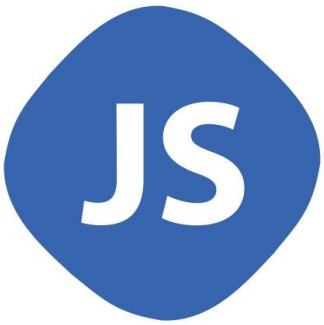


Объект XMLHttpRequest. Отправка данных

- GET и POST методы
- Отправка данных методом GET
- Отправка данных методом POST
- Комбинированная отправка данных
- Базовая аутентификация

GET и POST методы

Во время обычной отправки формы **браузер** собирает значения её полей, кодирует, делает из них строку и отправляет данные либо в адресной строке, либо в теле HTTP-запроса (зависит от метода – GET/POST).

При отправке данных через **XMLHttpRequest**, все эти действия **разработчик** должен делать **самостоятельно** в **JavaScript**-коде.

> **HTTP — это текстовый протокол**, поэтому все данные передаются в виде текста. Заголовки можно отделить друг от друга только переносом строки.

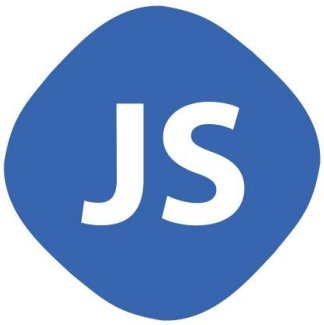
Впрочем, используя объекты **FormData** и **URLSearchParams** можно значительно упростить себе жизнь.

Метод GET

Метод GET используется клиентом при запросе к серверу по умолчанию. При выполнении запроса клиент и сервер обмениваются информацией в HTTP-заголовках запроса и ответа.

- **Запрос**. Используя **HTTP-заголовки** клиент сообщает серверу:
  - адрес ресурса (URL), который он хочет получить;





- версию протокола HTTP;
- MIME-типы документов, которые он поддерживает;
- версию и название клиентского программного обеспечения.

Тело HTTP-сообщения в запросе не передается.

- **Ответ**. Используя **HTTP-заголовки** сервер сообщает клиенту:
  - версию HTTP-протокола;
  - код возврата;
  - тип содержания тела ответа;
  - размер тела ответа;
  - ряд других необязательных директив HTTP-заголовка.

  Сам запрашиваемый ресурс, обычно это HTML-страница, передается в теле ответа.

> ***Важно***. Строка запроса (пары имя=значение) отправляется в URL-адрес запроса GET:
>
> `http://js/index.html?`**`name=value1&login=value2`**

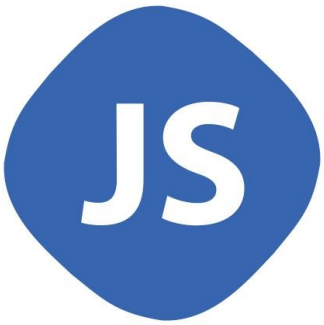
Некоторые другие заметки о запросах **GET**:

- GET запросы могут кэшироваться.
- GET запросы остаются в истории браузера.
- GET запросы могут быть закладками.
- GET запросы никогда не должны использоваться при работе с конфиденциальными данными.
- GET запросы имеют ограничения по длине.
- GET запросы должны использоваться только для извлечения данных.

Метод POST

Метод POST — это альтернатива методу GET. При выполнении запроса клиент и сервер обмениваются информацией в **HTTP-заголовках** запроса и ответа.

- **Запрос**. Используя **HTTP-заголовки** клиент сообщает серверу:



- адрес ресурса (URL), который он хочет получить;
- версию протокола HTTP;
- тип содержания тела сообщения;
- размер тела сообщения;
- MIME-типы документов, которые он поддерживает;
- версию и название клиентского программного обеспечения.

При обмене данными по методу POST в запросе клиента присутствует тело HTTP-сообщения. Это тело может формироваться из данных, которые вводятся в **HTML-форме**, или из присоединенного **внешнего файла**.

- **Ответ**. Используя **HTTP-заголовки** сервер сообщает клиенту:
  - версию HTTP-протокола;
  - код возврата;
  - тип содержания тела ответа;
  - размер тела ответа;
  - ряд других необязательных директив HTTP-заголовка.

  В ответе, как правило, присутствует **тело HTTP-сообщения**.

Важно. Строка запроса (пары имя=значение) отправляется в теле HTTP-сообщения запроса POST:

```
http://js/index.php HTTP/1.1
POST
Host: js
name=value1&login=value2
```

Некоторые другие заметки о запросах **POST**:

- POST запросы никогда не кэшируются.
- POST запросы не сохраняются в журнале обозревателя.
- POST запросы не могут быть закладками.
- POST запросы не имеют ограничений по длине данных.





Это общая информация о GET и POST. Теперь рассмотрим **реализацию обмена данными** между клиентом и сервером при разных методах.

Отправка данных методом GET

До сих пор мы отправляли на сервер HTTP-запрос и получали в ответ некоторые данные. Это могли быть только заголовки ответа или заголовки вместе с телом документа.

Но вместе с запросом на сервер можно передавать и **клиентские данные**.

> ***Важно***. Курс рассматривает **клиентскую** часть программирования приложений. Серверные файлы я прилагаю, но, если вы не обладаете навыками программирования в среде PHP, не беда. Ваша задача – разобраться с клиентом!
>
> Если заинтересуетесь языком PHP, курс **Программирование на языке PHP** вам в помощь.

Отправка простых текстовых данных

Усложним наши запросы. Отправим на сервер некоторые пользовательские данные, обработаем эти данные и отправим ответ клиенту.

Обратите внимание, данные на сервер передаются в **адресной строке**.

example_1. **index.html**

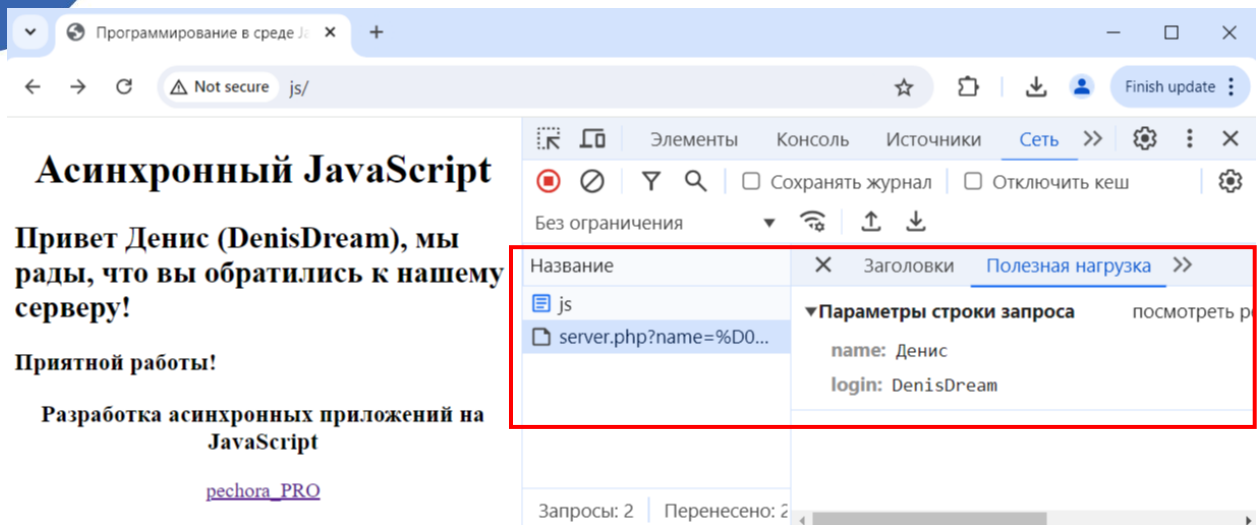
```
<div id="content"><!-- место для вставки контента --></div>
<script>
    // создаём объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    // инициализируем объект
```

```
    // к строке запроса добавили пересылаемые данные
    xhr.open('GET', 'server.php?name=Денис&login=DenisDream');
    // отправка запроса
    xhr.send();
    // сработает после получения ответа сервера
    xhr.onload = function() {
        // анализируем HTTP-статус ответа
        // если статус не 200, то произошла ошибка
        if (xhr.status != 200) {
            // выводим статус и текстовое сообщение ошибки
            alert(`Ошибка ${xhr.status}: ${xhr.statusText}`);
            // если status==200 выводим результат
        } else {
            console.log(`Готово, получили ${xhr.response.length}
            байт`);
            document.querySelector("#content").innerHTML =
            xhr.response;
        }
    };
</script>
```

example_1. server.php

```
<?php
    $name = $_GET['name'];
    $login = $_GET['login'];
    echo "<h2>Привет $name ($login), мы рады, что вы обратились к
    нашему серверу!</h2>";
```

```
echo "<h3>Приятной работы!</h3>";
```



> ***Примечание***. В ответе сервера присутствуют HTML-теги. Это сделано в учебных целях. Чаще всего ответ сервера будет содержать *чистые* данные, которые будут разбираться клиентским скриптом.

Отправка данных в формате JSON

При передаче более сложных структур данных можно в качестве значения ключа передавать **строку в формате JSON**. При этом не забудьте об **ограничениях на общий объем передаваемых данных**.

Обратите внимание, данные на сервер передаются в **адресной строке**.

example_2. index.html

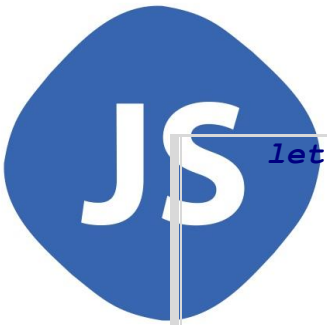
```
<div id="content"><!-- место для вставки контента --></div>

<script>

    // создаём объект XMLHttpRequest

    let xhr = new XMLHttpRequest();

    // определим объект с данными о товаре
```



```
let order = {
    id : 1,
    title : "Утепленная стеганая куртка",
    brand : "MARCO DI RADI",
    colour : "Синий",
    country : "Китай",
    hood : "Есть",
    size : "52",
    price : 25.590,
    image : "steganaya-marco.jpg"
};
// упаковываем объект с данными для отправки
JSONorder = JSON.stringify(order);
// инициализируем объект XHR
// JSON данные цепляем к адресной строке
xhr.open('GET', 'server.php?data=' + JSONorder);
// отправка запроса
xhr.send();
// сработает после получения ответа сервера
xhr.onload = function() {
    // анализируем HTTP-статус ответа
    // если статус не 200, то произошла ошибка
    if (xhr.status != 200) {
        // выводим статус и текстовое сообщение ошибки
        alert(`Ошибка ${xhr.status}: ${xhr.statusText}`);
        // если status==200 выводим результат
```

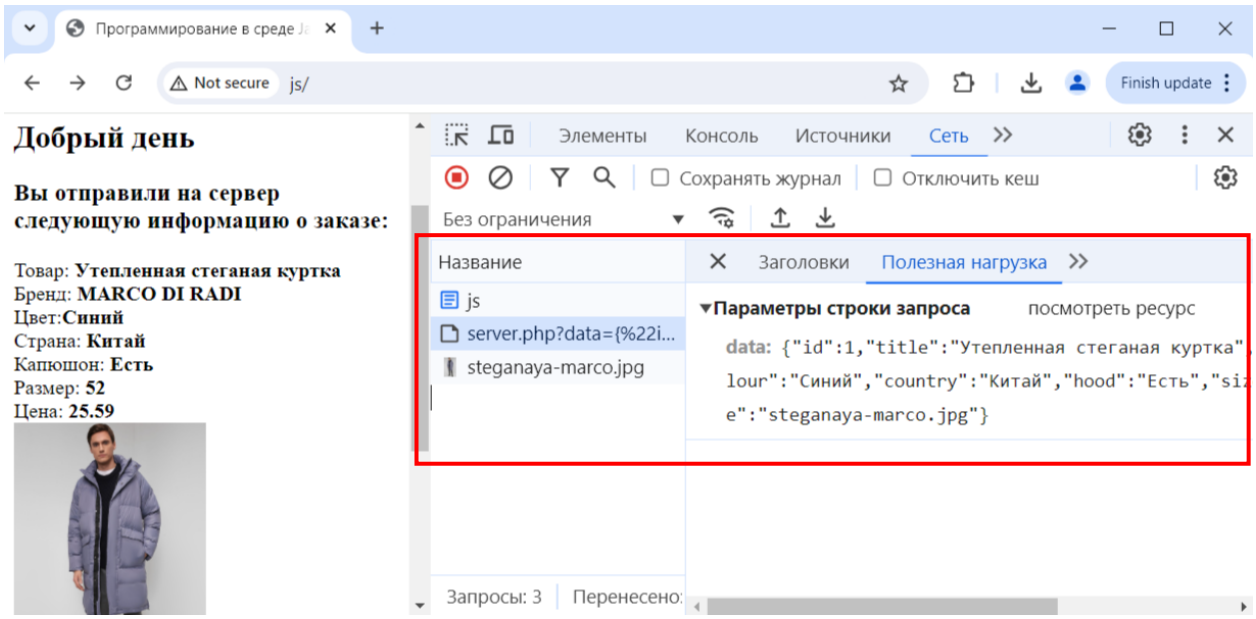


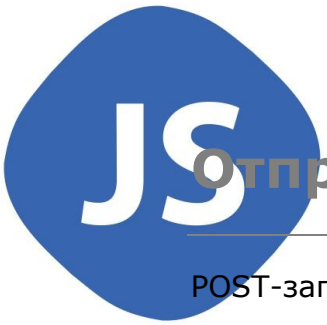
```
        } else {
            console.log(`Готово, получили ${xhr.response.length}
            байт`);
            document.querySelector("#content").innerHTML =
            xhr.response;
        }
    };
</script>
```

example_2. server.php

```
<?php
    // декодируем JSON формат в массив PHP
    $order = json_decode($_GET['data'], true);
    // формируем строку для отправки клиенту
    $out = sprintf("
        <h2>Добрый день</h2>
        <h3>Вы отправили на сервер следующую информацию о
        заказе:</h3>
        Товар: <b>%s</b> <br />
        Бренд: <b>%s</b> <br />
        Цвет:<b>%s</b> <br />
        Страна: <b>%s</b> <br />
        Капюшон: <b>%s</b> <br />
        Размер: <b>%s</b> <br />
        Цена: <b>%s</b> <br />
        <img src='images/%s' width='150px'>
        ",
```

```
        $order['title'],
        $order['brand'],
        $order['colour'],
        $order['country'],
        $order['hood'],
        $order['size'],
        $order['price'],
        $order['image']
    );
    // отправляем данные клиенту
    echo $out, "<h2>Спасибо за заказ!</h2>";
```





Отправка данных методом POST

POST-запросы используют более широкий спектр форматов для обмена данными. Указать **тип передаваемых данных** можно с помощью метода **setRequestHeader**().

> ***Примечание***. Как бы глубоко вы не вникали в POST-отправку данных, запомнить все нюансы очень сложно (откройте этот урок через месяц;).
> **Совет**: выполните примеры, создайте для себя краткий справочник по использованию метода. По мере необходимости обращайтесь к страницам справочника.

Формат **application/x-www-form-urlencoded** — это формат, в котором данные формы кодируются в виде пар **имя-значение**, **аналогично** строке запроса URI.

Этот формат используется по умолчанию для POST.

Формат **multipart/form-data** используется в HTML-формах и служит для корректной отправки **файлов на сервер** при работе с полями <input type="file">.

Этот тип кодирования подходит для передачи бинарных данных и файлов больших информационных объемов, включая изображения и видео.

Формат **text/plain** – используется для передачи данных в формате обычного текста. В современной веб-разработке используется редко.

Формат **application/json** — это формат, наиболее часто используемый для JSON-данных. Указывает на то, что отправляемый или получаемый контент — **строки формата JSON**.

При создании веб-приложений, которые обмениваются JSON-данными, рекомендуется использовать **application/json** в качестве типа контента.

Формат application/x-www-form-urlencoded





Формат **application/x-www-form-urlencoded** для метода POST используется по умолчанию. При использовании формата важно указать **тип передаваемых данных**.

```
// тип передаваемых данных
xhr.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded');
```

При отправке данных методом POST в запросе клиента присутствует тело **HTTP-сообщения**.

```
// отправка запроса
// к запросу добавляем тело сообщения
xhr.send(data);
```

example_3. **index.html**

```
<script>
    let xhr = new XMLHttpRequest();
    xhr.open ("POST", "server.php");
    // тип передаваемых данных
    xhr.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded');
    // формируем отправляемые в теле запроса данные
    data = "login=DenisDream&email=denis-dream@gmail.com";
    // отправка запроса
    // к запросу добавляем тело сообщения
    xhr.send(data);
    // обработчик ответа сервера
    xhr.onload = function() {
        console.log(xhr.response);
```

```
}
</script>
```

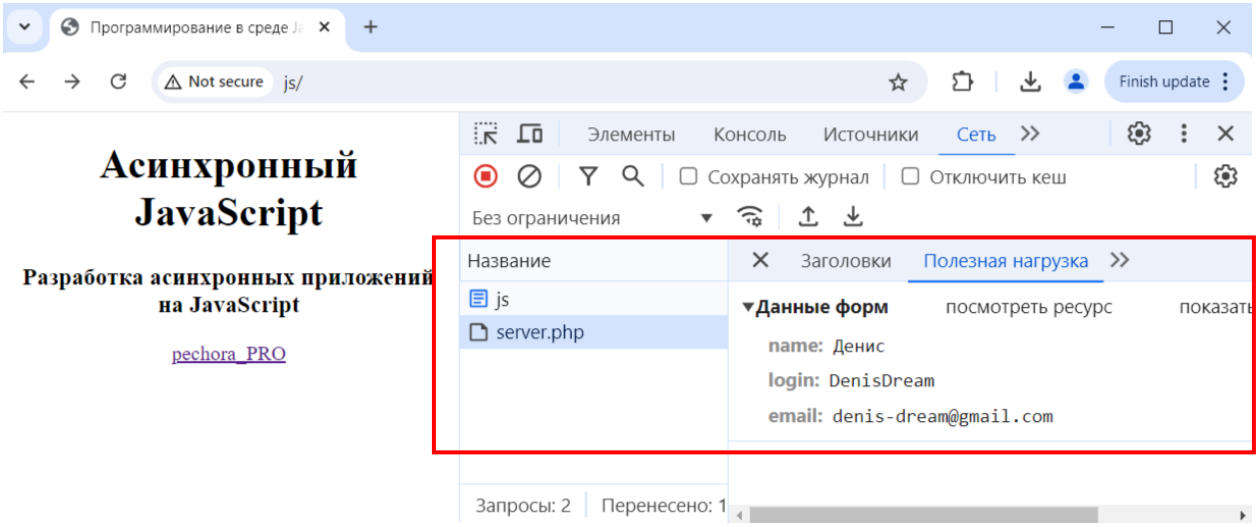
example_3. **server.php**

```
<?php

    // отправим пользователю массив $_POST

    print_r ($_POST);
```

В разделе **Полезная нагрузка** находится вкладка **Данные форм**. Обратите внимание, никаких форм в файле, отправляющем данные нет, есть сформированное в коде **тело сообщения**.



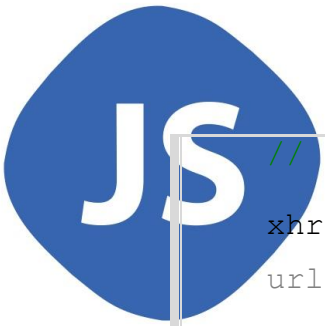
Если в примере **example_3** в качестве значений ключей мы использовали простые **текстовые данные**, то теперь, в качестве значения используется **строка JSON** формата.

example_4. **index.html**

```
<script>

    let xhr = new XMLHttpRequest();

    xhr.open ("POST", "server.php");
```



```
// тип передаваемых данных
xhr.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded');
// определим объект с данными о товаре
let order = {
    id : 1,
    title : "Утепленная стеганая куртка",
    brand : "MARCO DI RADI",
    colour : "Синий",
    country : "Китай",
    hood : "Есть",
    size : "52",
    price : 25.590,
    image : "steganaya-marco.jpg"
};
// упаковываем заказ в JSON формат
JSONorder = JSON.stringify(order);
// в качестве значения ключа используется строка JSON формата
data = `order=${JSONorder}`;
// отправка запроса
// к запросу добавляем тело сообщения
xhr.send(data);
// сработает после получения ответа сервера
xhr.onload = function() {
    console.log(xhr.response);
    console.log(JSON.parse(xhr.response));
```



```
}
</script>
```

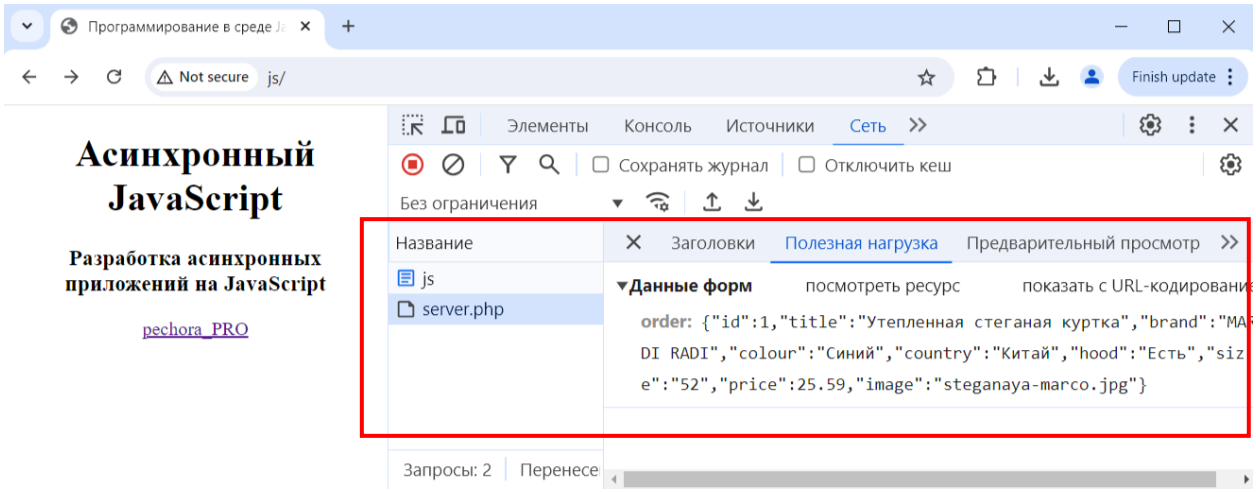
example_4. server.php

```
<?php

    // отправляем данные о заказе обратно клиенту

    print_r ($_POST['order']);
```

Тело сообщения состоит из ключа **order** и значения – **строки в формате JSON**.



Усложним структуру отправляемых на сервер данных. **Объединим данные двух массивов**.

example_5. index.html

```
<script>

    let xhr = new XMLHttpRequest();

    xhr.open ("POST", "server.php");

    // тип передаваемых данных

    xhr.setRequestHeader('Content-Type', 'application/x-www-form-
    urlencoded');
```

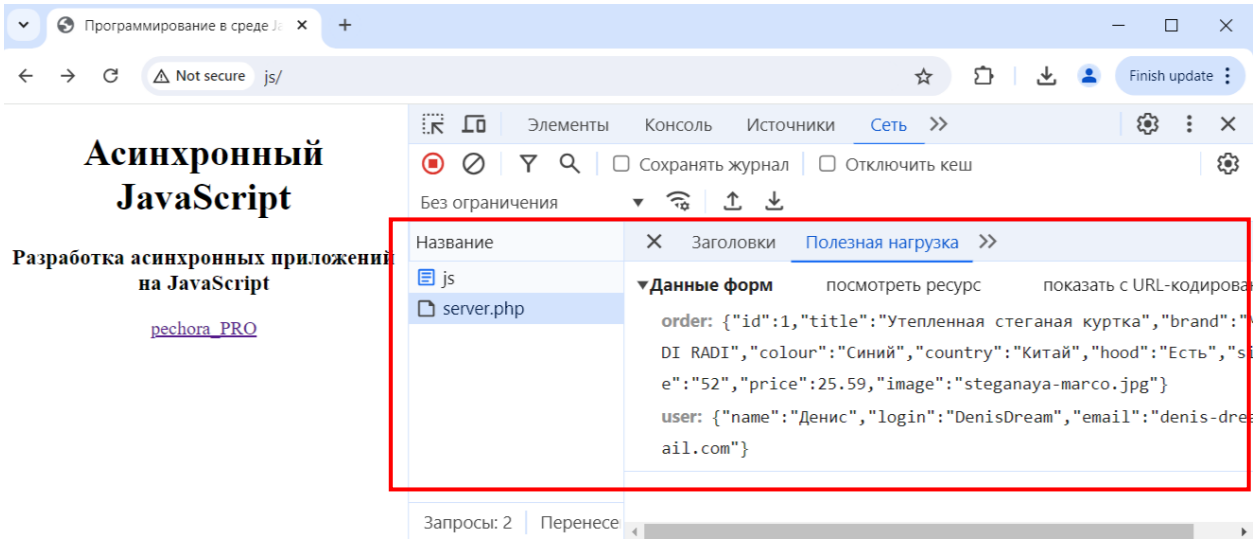
```
// определим объект с данными о товаре
let order = {
    id : 1,
    title : "Утепленная стеганая куртка",
    brand : "MARCO DI RADI",
    colour : "Синий",
    country : "Китай",
    hood : "Есть",
    size : "52",
    price : 25.590,
    image : "steganaya-marco.jpg"
};
// определим объект с данными о пользователе
let user = {
    name : "Денис",
    login : "DenisDream",
    email : "denis-dream@gmail.com"
};
// упаковываем заказ в JSON формат
JSONorder = JSON.stringify(order);
// упаковываем пользователя в JSON формат
JSONuser = JSON.stringify(user);
// формируем строку для передачи на серввер
// в качестве значений ключей используются строки JSON формата
data = `order=${JSONorder}&user=${JSONuser}`;
```

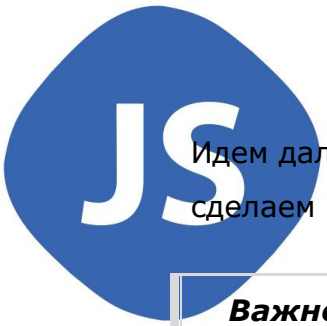
```
    // отправка запроса
    // к запросу добавляем тело сообщения
    xhr.send(data);
    // сработает после получения ответа сервера
    xhr.onload = function() {
        console.log(xhr.response);
    }
</script>
```

example_5. server.php

```
<?php
    // отправляем данные обратно клиенту
    // данные о заказе
    print_r ($_POST['order']);
    // данные о клиенте
    print_r ($_POST['user']);
```

Тело сообщения состоит из ключей **order**, **user** и их значений – **строк в формате JSON**.





Идем дальше. В следующем примере сбор данных для отправки на сервер сделаем красиво ;)

> ***Важно.*** Для сбора данных в демонстрационном примере **example_6** используем объект **URLSearchParams**.

При использовании объекта **URLSearchParams** тип передаваемых данных указывать **не нужно**.

example_6. **index.html**

```
<script>
    // определим объект с данными о товаре
    let order = {
        id : 1,
        title : "Утепленная стеганая куртка",
        brand : "MARCO DI RADI",
        colour : "Синий",
        country : "Китай",
        hood : "Есть",
        size : "52",
        price : 25.590,
        image : "steganaya-marco.jpg"
    };
    // создадим объект для сбора данных
    data = new URLSearchParams();
    // сохраним данные в формате Имя-Значение
    for(val in order){
        data.append(val, order[val])
    }
```



```
    let xhr = new XMLHttpRequest();
    xhr.open ("POST", "server.php");
    // отправка запроса
    // к запросу добавляем тело сообщения
    xhr.send(data);
    // сработает после получения ответа сервера
    xhr.onload = function() {
        console.log(xhr.response);
    }
</script>
```

example_6. server**.php**

```
<?php
    // отправляем данные о заказе обратно клиенту
    print_r ($_POST);
```

Формат text/plain

Используем данные формата **text/plain**.

```
// при использовании текстового формата тип передаваемых данных можно не указывать
// данные передаем в текстовом формате
xhr.setRequestHeader('Content-type', 'text/plain');
```

example_7. **index.html**

```
<script>
    let xhr = new XMLHttpRequest();
```

```
    xhr.open ("POST", "server.php");
    // определим объект с данными о товаре
    let order = {
        id : 1,
        title : "Утепленная стеганая куртка",
        brand : "MARCO DI RADI",
        colour : "Синий",
        country : "Китай",
        hood : "Есть",
        size : "52",
        price : 25.590,
        image : "steganaya-marco.jpg"
    };
    // упаковываем объект с данными для отправки
    JSONalbum = JSON.stringify(order);
    // отправка запроса
    // к запросу добавляем тело сообщения
    // отправляем данные текстового формата
    xhr.send(JSONalbum);
    // сработает после получения ответа сервера
    xhr.onload = function() {
        console.log(xhr.response);
    }
</script>
```

example_7. server.php

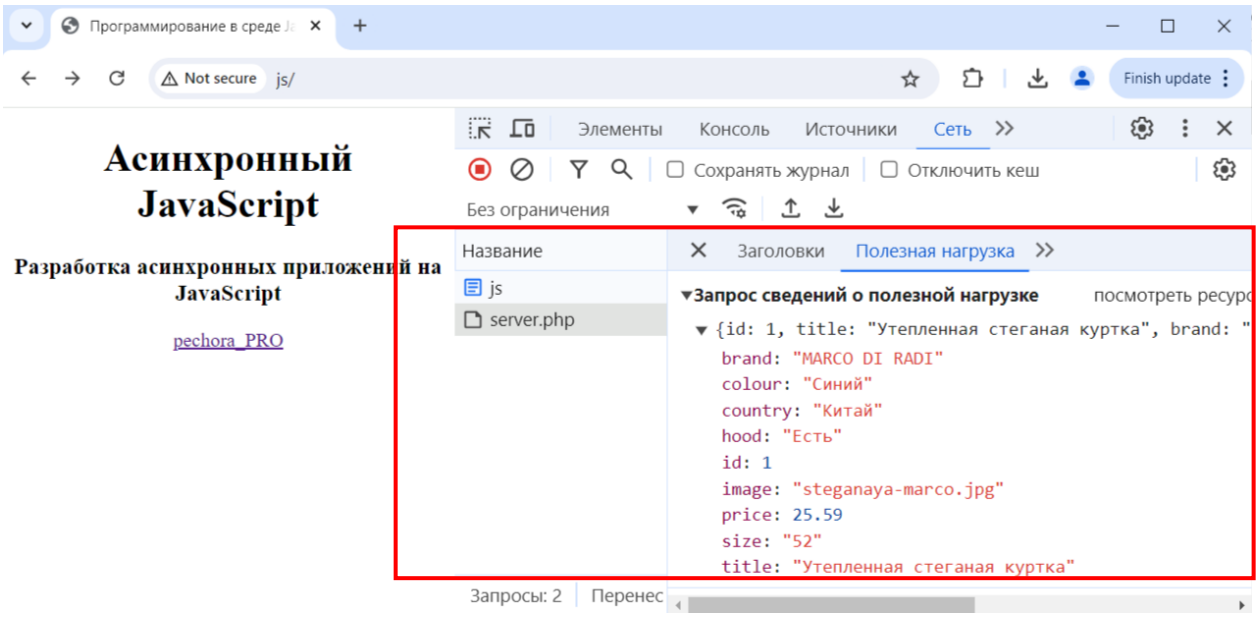
```php
<?php

    // извлекаем и преобразуем в PHP-массив полученные данные

    $data = file_get_contents("php://input");

    // отправляем данные обратно клиенту

    print_r ($data);
```



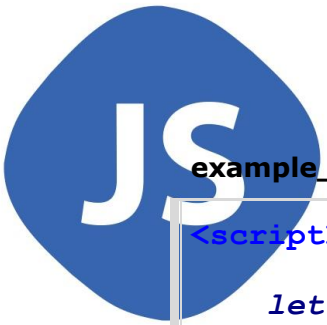
Формат application/json

В следующем примере для отправки на сервер используются данные формата **application/json**.

```
// при использовании текстового формата тип передаваемых данных можно не указывать

// данные передаем в формате JSON

xhr.setRequestHeader('Content-type', 'application/json');
```



example_8. index.html

```
<script>
    let xhr = new XMLHttpRequest();
    xhr.open ("POST", "server.php");
    // определим объект с данными о товаре
    let order = {
        id : 1,
        title : "Утепленная стеганая куртка",
        brand : "MARCO DI RADI",
        colour : "Синий",
        country : "Китай",
        hood : "Есть",
        size : "52",
        price : 25.590,
        image : "steganaya-marco.jpg"
    };
    // упаковываем объект с данными для отправки
    JSONalbum = JSON.stringify(order);
    // отправка запроса
    // к запросу добавляем тело сообщения
    // отправляем данные формата JSON
    xhr.send(JSONalbum);
    // сработает после получения ответа сервера
    xhr.onload = function() {
        console.log(xhr.response);
    }
```



```
</script>
```

example_8. server.php

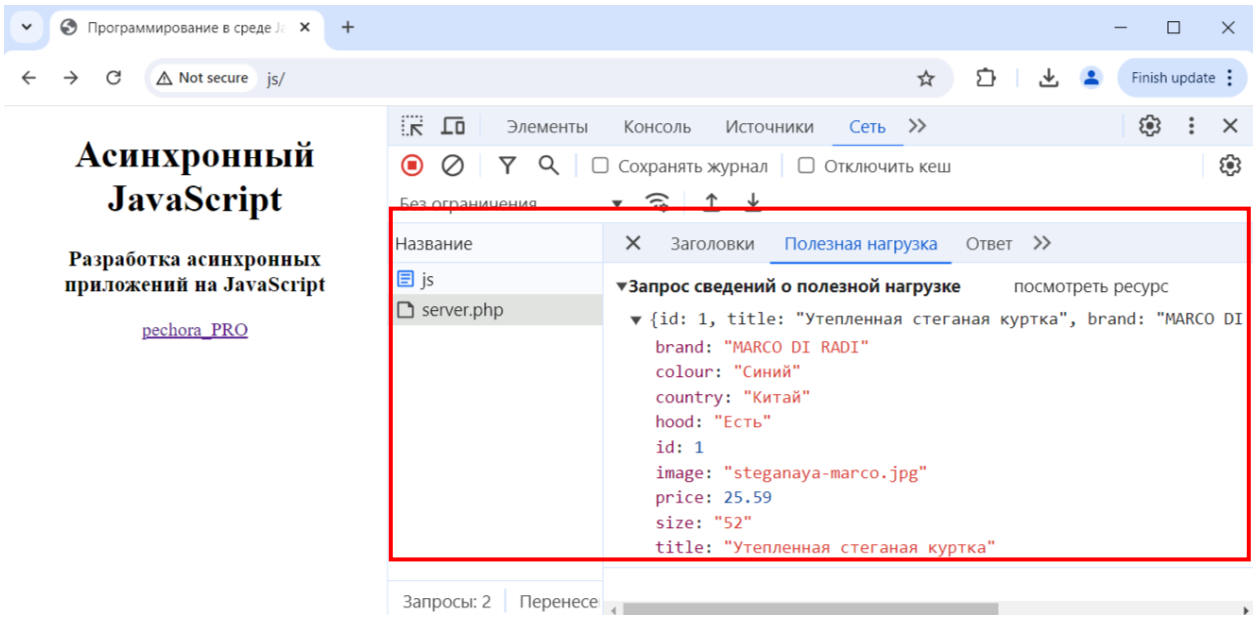
```
<?php

    // извлекаем и преобразуем в PHP-массив полученные данные

    $data = json_decode(file_get_contents("php://input"), true);

    // отправляем данные обратно клиенту

    print_r ($data);
```

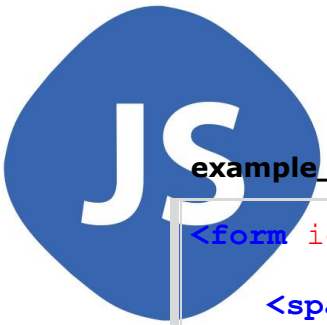


Формат multipart/form-data

В следующих примерах для передачи на сервер используются данные формата **multipart/form-data.**

> ***Важно.*** Для сбора данных используем объект **FormData**.

При использовании объекта **FormData** тип передаваемых данных указывать **не нужно**.



example_9. index.html

```
<form id="frm" onsubmit="return fnSubmit()">
    <span>Логин:</span>
    <input type="text" name="login" value="DenisDream"><p>
    <span>Пароль:</span>
    <input type="text" name="pwd" value="qwerty@"><p>
    <button>Отправить</button>
</form>
<script>
    function fnSubmit() {
        let xhr = new XMLHttpRequest();
        xhr.open("POST", "server.php");
        // укажем ожидаемый тип ответа
        xhr.responseType = "json";
        // при использовании FormData тип передаваемых данных
        указывать не надо
        // определим объект FormData
        let data = new FormData(document.getElementById("frm"));
        // отправляем данные
        xhr.send(data);
        // сработает после получения ответа сервера
        xhr.onload = function() {
            console.log(xhr.response);
        }
        return false;
    }
```



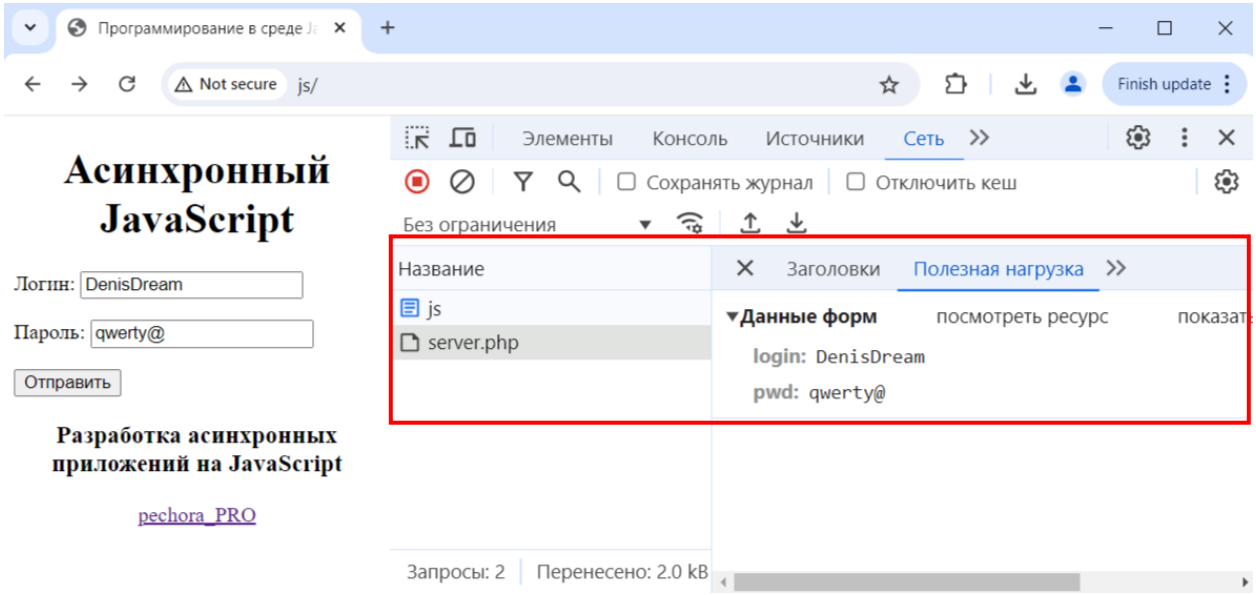
```
</script>
```

example_9. server.php

```
<?php

    // отправим пользователю текстовые данные формы

    echo json_encode($_POST);
```



Пересылаем двоичные файлы

Но самое интересное, это отправка на сервер **двоичных** файлов.

example_10. index.html

```
<form id="frm" onsubmit="return fnSubmit()">

    <span>Прикрепите файл:</span><p>

    <input type="file" name="file"><p>

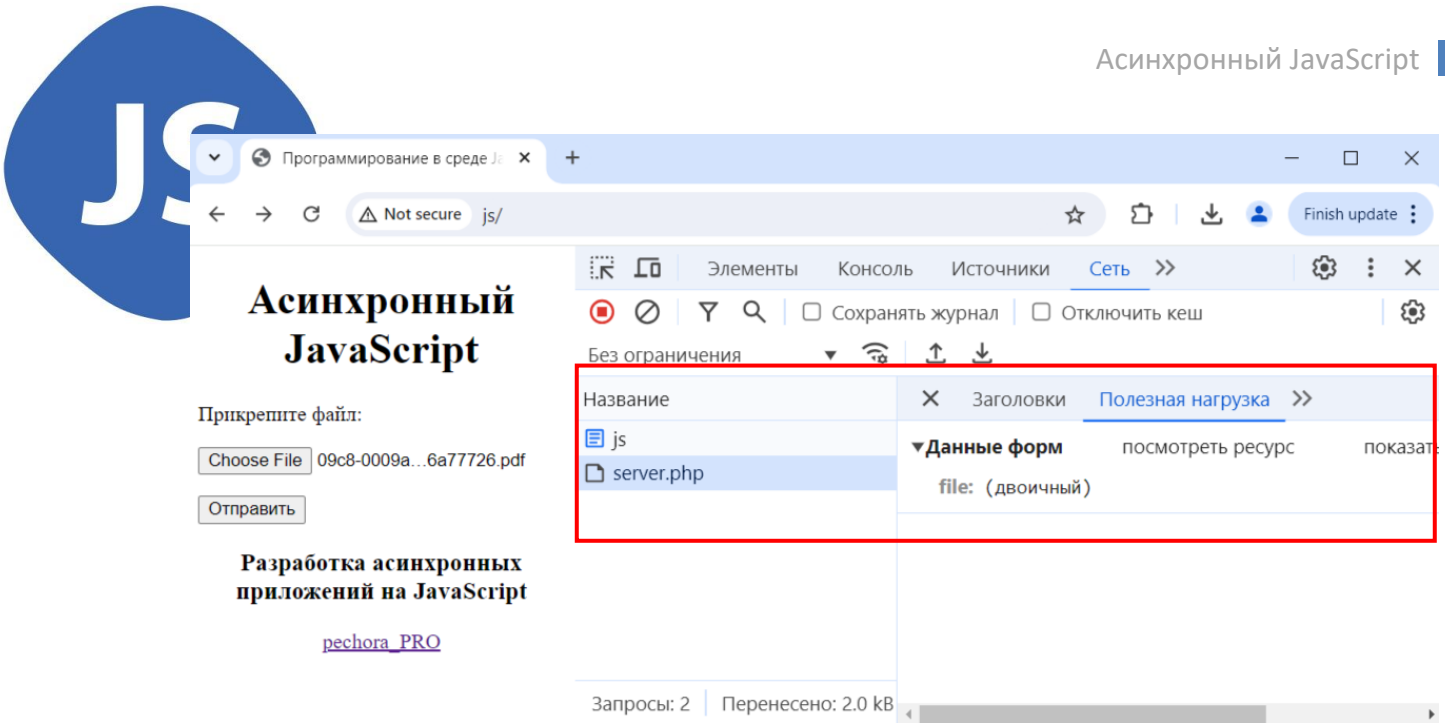
    <button>Отправить</button>

</form>
```

```
<script>
    function fnSubmit() {
        let xhr = new XMLHttpRequest();
        // при использовании FormData тип передаваемых данных
        указывать не надо
        // определим объект FormData
        form_data = new FormData(document.getElementById('frm'));
        xhr.open("POST", "server.php");
        // отправляем данные
        xhr.send(form_data);
        // сработает после получения ответа сервера
        xhr.onload = function() {
            console.log(xhr.response);
        }
        return false;
    }
</script>
```

example_10. server.php

```
<?php
    // отправим пользователю
    // информацию о загруженном файле
    echo json_encode($_FILES["file"]);
```

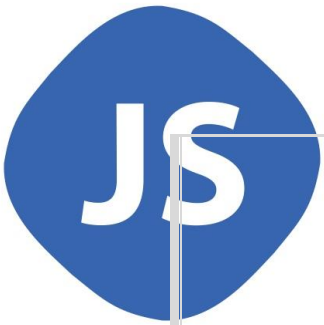


Пересылаем текстовые и двоичные файлы

В демонстрационном примере **example_11** форма содержит как **текстовые,** так и **двоичные** данные.

example_11. index.html

```
<form id="frm" onsubmit="return fnSubmit()">
    <span>Логин:</span>
    <input type="text" name="login" value="DenisDream"><p>
    <span>Пароль:</span>
    <input type="text" name="pwd" value="qwerty@"><p>
    <span>Прикрепите файл:</span><p>
    <input type="file" name="file"><p>
    <button>Отправить</button>
</form>
<script>
    function fnSubmit() {
        let xhr = new XMLHttpRequest();
```

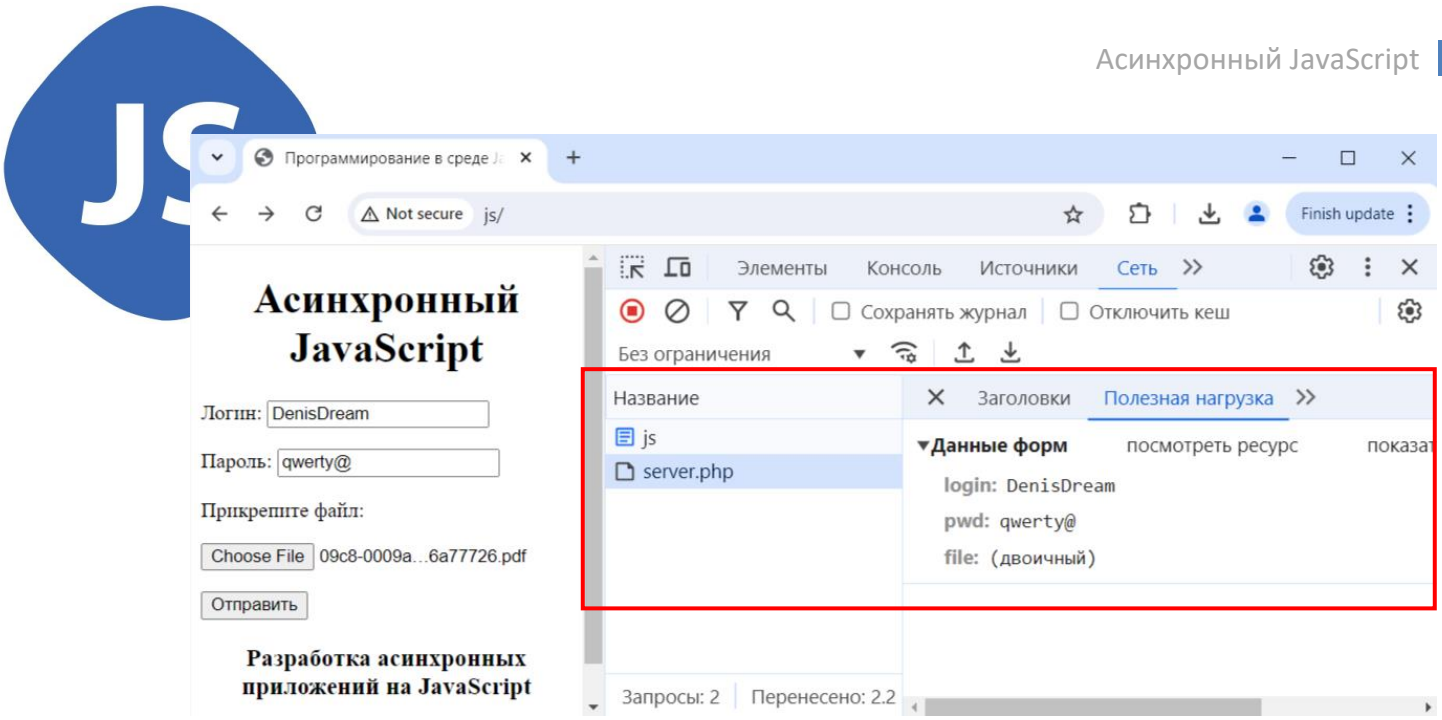


```
        // при использовании FormData тип передаваемых данных
        указывать не надо
        // определим объект FormData
        data = new FormData(document.getElementById("frm"));
        xhr.open("POST", "server.php");
        // отправляем данные
        xhr.send(data);
        // сработает после получения ответа сервера
        xhr.onload = function() {
            console.log(xhr.response);
        }
        return false;
    }
</script>
```

example_11. server.php

```
<?php
    // отправим пользователю
    // текстовые данные формы
    $resp['data'] = $_POST;
    // информацию о загруженном файле
    $resp['file'] = $_FILES["file"];
    echo json_encode($resp);
```



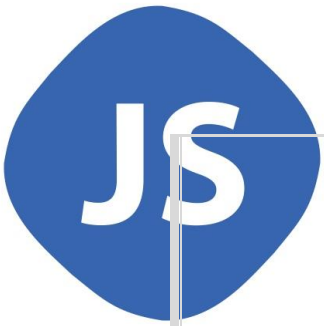


Комбинированная отправка данных

Ничто не мешает нам отправить данные на сервер как в **адресной строке**, так и в **теле HTTP-запроса**.

example_12. index.html

```
<form id="frm" onsubmit="return fnSubmit()">
    <span>Логин:</span>
    <input type="text" name="login" value="DenisDream"><p>
    <span>Пароль:</span>
    <input type="text" name="pwd" value="qwerty@"><p>
    <span>Прикрепите файл:</span><p>
    <input type="file" name="file"><p>
    <button>Отправить</button>
</form>
<script>
    function fnSubmit() {
```



```
        let xhr = new XMLHttpRequest();
        // при использовании FormData тип передаваемых данных
        указывать не надо
        // определим пустой объект FormData
        data = new FormData();
        // добавим к объекту FormData двоичный файл
        data.append("file",
        document.querySelector('[name="file"]').files[0])
        // добавим в адресную строку GET-параметры
        $get = `?
        login=${document.querySelector('[name="login"]').value}&
        pwd=${document.querySelector('[name="pwd"]').value}`;
        // инициализируем объект
        xhr.open("POST", "server.php" + $get);
        // отправляем данные
        // # GET - адресная строка
        // # POST - тело сообщения
        xhr.send(data);
        // сработает после получения ответа сервера
        xhr.onload = function() {
            console.log(xhr.response);
        }
        return false;
    }
</script>
```



example_12. server.php

```
<?php

    // отправим пользователю

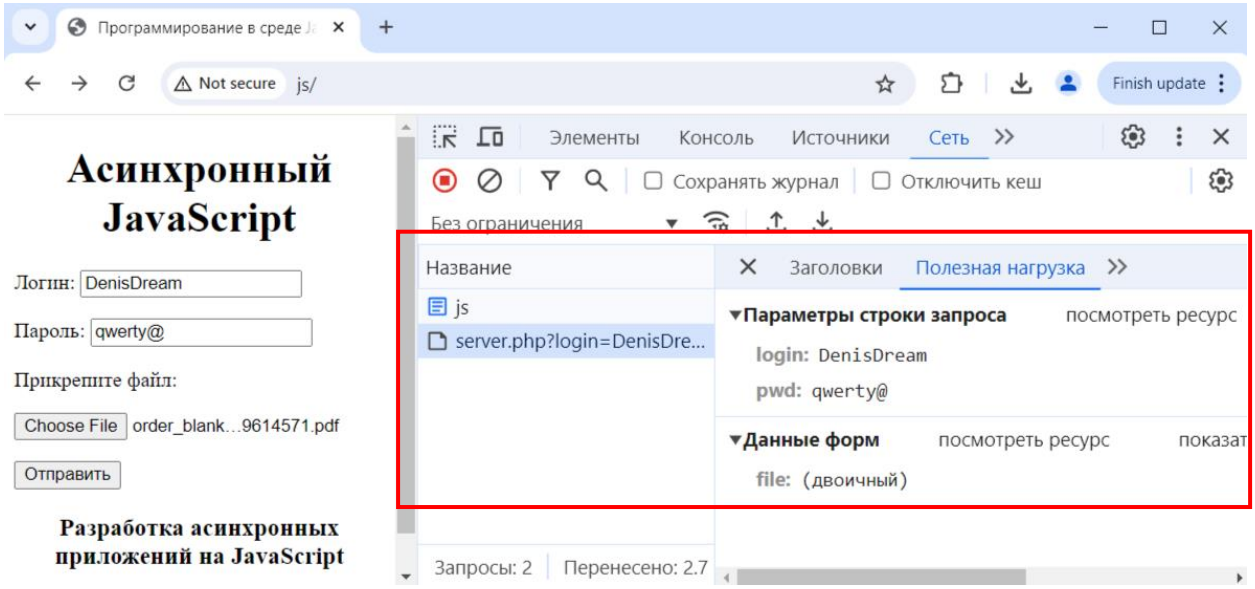
    // текстовые данные формы

    $resp['data'] = $_GET;

    // информацию о загруженном файле

    $resp['file'] = $_FILES["file"];

    echo json_encode($resp);
```



Базовая HTTP-аутентификация

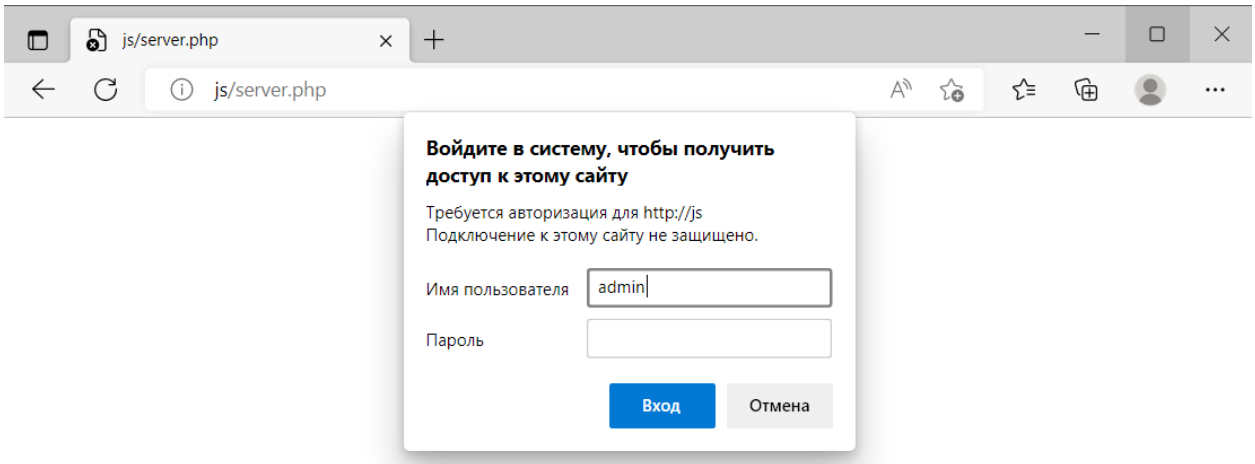
Если вы никогда не сталкивались с базовой (стандартной) аутентификацией прежде, чем писать асинхронный код, попробуйте ее на простом примере.

В директории **example_13** находится файл **server.php**, использующий базовую аутентификацию. Переместите файл на сервер, перейдите к файлу по **прямой ссылке**.

У меня адрес ссылки выглядит следующим образом:

http://js/server.php

Диалоговое окно запроса имени и пароля пользователя – это и есть стандартная HTTP-аутентификация. Внешний вид процесса аутентификации продемонстрирован на рисунке.

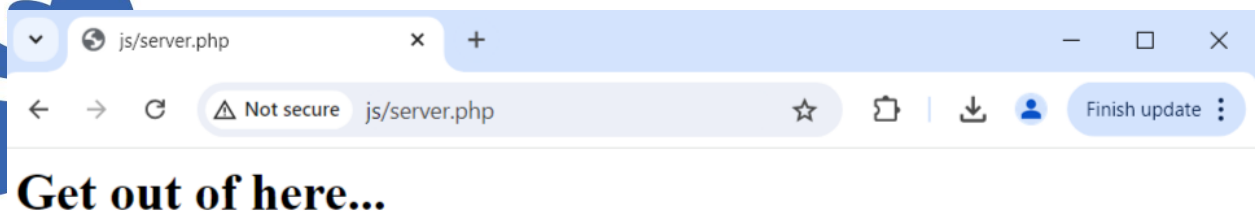


Протестируйте работу скрипта, для **прохождения аутентификации** используйте следующие данные:

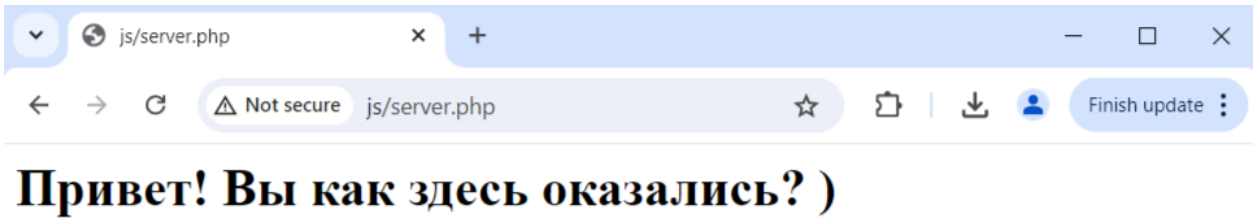
- Имя пользователя: **admin**
- Пароль: **12345**

> ***Примечание***. После ввода данных в форму, данные о пользователе будут сохранены в браузере. Для повторного вызова диалогового окна аутентификации закройте и снова откройте браузер, либо в настройках браузера выполните команду - Удалить данные о просмотре веб-страниц: **Файлы cookie и другие данные сайтов**.

Вывод приложения при **неудачной** аутентификации.



Вывод приложения при **удачной** аутентификации.



Вернемся к асинхронным запросам. При реализации запросов к серверу, требующего выполнение стандартной HTTP-аутентификации, задействуем необязательные параметры метода **open** объекта **XHR**.

```
// инициализируем объект
xhr.open('GET', 'server.php', true, 'admin', '12345');
```

где:

- **admin** - имя пользователя;
- **12345** - пароль пользователя.

example_13. **index.html**

```
<script>
    // создаём новый XMLHttpRequest-объект
    let xhr = new XMLHttpRequest();
    let user = 'admin';
    let pwd = '12345';
    // инициализируем объект с указанием имени и пароля
```

```
    пользователя
    xhr.open('GET', 'server.php', true, user, pwd);
    // отправка запроса
    xhr.send();
    // сработает после получения ответа сервера
    xhr.onload = function() {
        // ответ парсится автоматически
        let resp = xhr.response;
        // вставляем строку в документ
        document.querySelector("#content").innerHTML = resp;
    };
</script>
```

В демонстрационном примере **server.php** представлена серверная часть приложения базовой аутентификации.

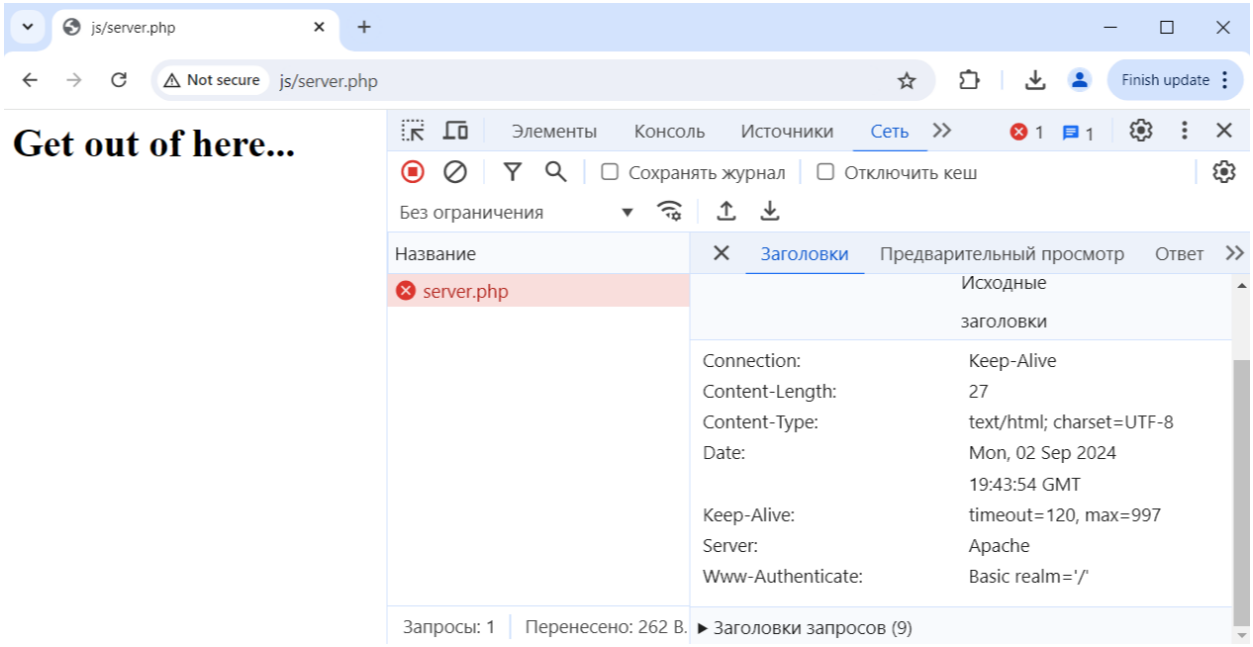
example_13. server.php

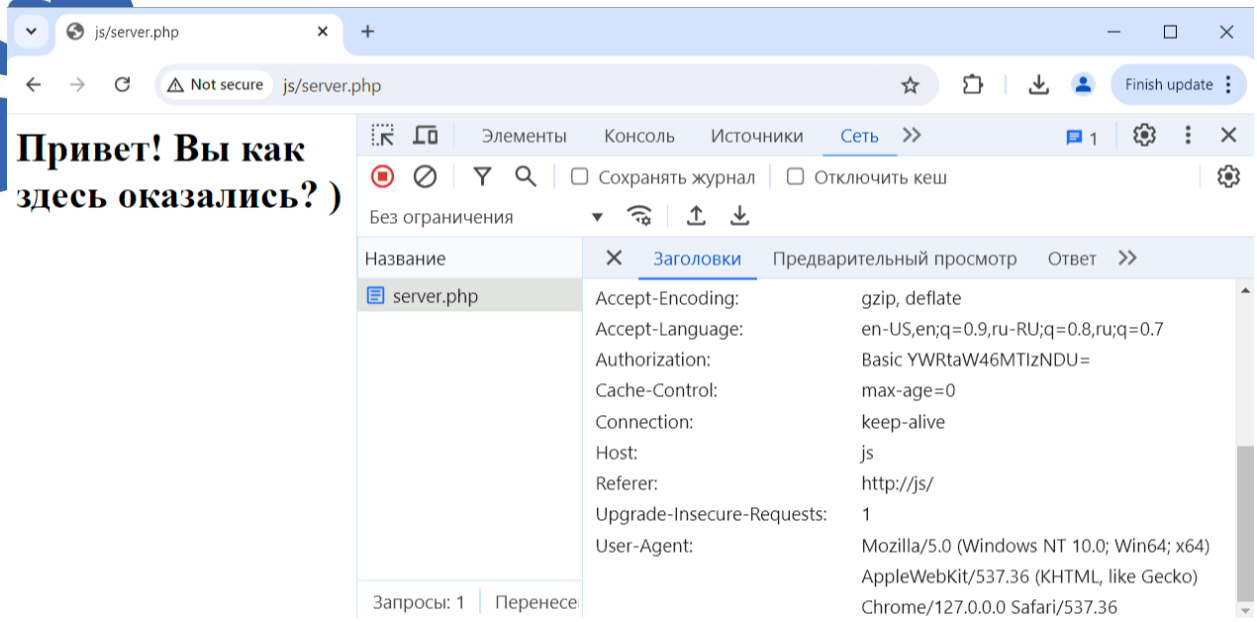
```
<?php
    # серверная часть базовой аутентификации
    if ((!isset($_SERVER['PHP_AUTH_USER'])) ||
        (!isset($_SERVER['PHP_AUTH_PW'])) ||
        ($_SERVER['PHP_AUTH_USER'] != 'admin') ||
        ($_SERVER['PHP_AUTH_PW']) != '12345')
    {
        header("HTTP/1.1 401 Unauthorized");
        header("WWW-Authenticate: Basic realm='/'");
        exit('<h1>Get out of here...</h1>');
```

```
}
else {
    echo "<h1>Привет! Вы как здесь оказались? )</h1>";
}
```

Заголовки базовой HTTP-аутентификации

Проанализируйте **заголовки** при выполнении как успешной, так и ошибочной аутентификации пользователя в системе.





Дополнительные задания в **самостоятельных работах**:

- Выполнение асинхронного запроса с передачей данных в адресной строке браузера.
- Выполнение асинхронного запроса с передачей данных в теле запроса. Использование возможности объекта URLSearchParams.
- Создание программы асинхронной авторизации пользователя с переадресацией.
- Выполнение асинхронного запроса с передачей данных формата JSON в теле запроса.

P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).



К каждой **теме** курса:

- **Практические работы**.

К **курсу**:

- **Дополнительный практикум.**

